

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	30 June 2026
Team ID	[Team ID]
Project Name	Customer Care Registry
Maximum Marks	4 Marks

Technical Architecture

The Customer Care Registry follows a 3-tier architecture: a React single-page application (client), an Express REST API (application/server layer), and a JSON-file-based data store (data layer), with the Google Gemini API integrated as an external AI service for complaint categorization and reply suggestions.

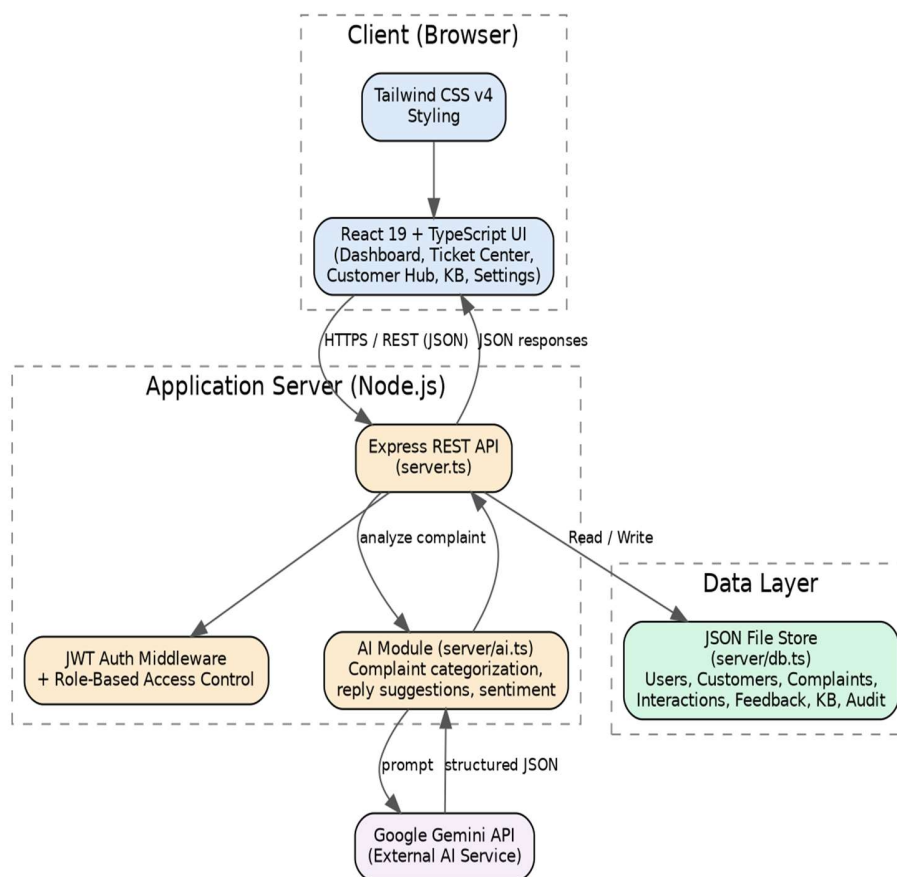


Figure 1: Technical architecture of the Customer Care Registry

Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Role-based web dashboard: Analytics Dashboard, Ticket Center, Customer Hub, Knowledge Base, Audit Logs, System Settings	React 19, TypeScript, Tailwind CSS v4, Vite, lucide-react icons, recharts, motion
2	Application Logic — API Server	REST API handling auth, customers, complaints, interactions, feedback, KB, analytics, audit, settings	Node.js, Express 4, TypeScript (tsx runtime)
3	Application Logic — Auth & RBAC	JWT-based authentication and role-based access control middleware (Super Admin, Support Manager, Support Agent, Customer)	JSON Web Tokens (JWT), Express middleware

S.No	Component	Description	Technology
4	Application Logic — AI Module	Auto-categorizes complaints (category, priority, department), suggests agent replies, derives feedback sentiment	Google Gemini API (@google/genai), keyword-heuristic fallback
5	Database	Stores users, customers, complaints, interactions, feedback, KB articles, audit logs, settings	JSON file-based data store (server/db.ts), upgradeable to MongoDB / PostgreSQL
6	File Storage	Complaint and interaction attachments	Base64-encoded inline storage (local filesystem / object storage ready)
7	External API	AI-powered text analysis for complaint categorization and reply drafting	Google Gemini API (gemini model family)
8	Machine Learning / AI Model	Natural-language classification of complaint text into category, priority, and department; sentiment detection	Gemini generative model with structured JSON output (Type schema)
9	Infrastructure (Server / Build)	Local development and production build/deploy	Vite (dev/build), esbuild (server bundle), Node.js runtime; deployable to any Node-compatible host or container

Table-2: Application Characteristics

Characteristics	Description	Technology
Open-Source Frameworks	React, Express, Vite, Tailwind CSS, TypeScript compiler tooling	React 19, Express 4, Vite 6
Security Implementations	JWT-based session tokens, per-route role checks (requireRole), server-side validation on all write endpoints, audit trail of user actions	JWT, RBAC middleware
Scalable Architecture	3-tier architecture: presentation (React SPA) → application (Express REST API) → data (pluggable store), so the data layer can scale independently of the UI	3-Tier Architecture
Availability	Stateless API layer that can be run as multiple instances behind a load balancer; front end served as static assets from a CDN-capable host	Node.js clustering / Load Balancer ready
Performance	Vite-optimized production bundles, on-demand analytics computation, and a lightweight JSON data layer suited to fast reads for moderate data volumes	Vite build optimization, in-memory analytics aggregation